

Supplementary Material for Implementation

AI teams

School of Mathematics and Physics
Xi'an Jiaotong-Liverpool University

May 5, 2026

Data Preparation: Overview

Before implementing a neural network, we need to prepare the dataset carefully.

- Load the dataset from a file or package
- Read the dataset description or instructions
- Extract inputs/features as X
- Extract outputs/labels/targets as y

Loading Data from Different Sources

Datasets can come from many sources.

From a NumPy file

```
import numpy as np

data = np.load("data.npy")
```

From Excel or CSV using pandas

```
import pandas as pd

df_excel = pd.read_excel("data.xlsx")
df_csv = pd.read_csv("data.csv")
```

Loading Built-in Datasets

Some Python packages provide ready-to-use datasets.

Example: scikit-learn

```
from sklearn.datasets import load_iris

dataset = load_iris()

X = dataset.data
y = dataset.target
```

Here, X contains the input features, and y contains the target labels.

Read the Dataset Description

Before using a dataset, always read its documentation.

- What does each feature mean?
- What is the output or target variable?
- Is the task classification or regression?
- Are the features numerical or categorical?
- Are the data already normalized or standardized?

Good neural network implementation starts with understanding the data.

Extracting Inputs and Outputs

For supervised learning, we usually separate the dataset into:

X = input features

y = output labels or target values

Each row of X should correspond to one sample.

Each row of y should correspond to the output for the same sample.

Shape of the Input Matrix

The input matrix X is usually arranged as:

$$X \in \mathbb{R}^{n \times d}$$

where:

- n = number of samples
- d = number of input features

$$X = \begin{bmatrix} - & x_1^T & - \\ - & x_2^T & - \\ & \vdots & \\ - & x_n^T & - \end{bmatrix}$$

Each row is one data sample.

Shape of the Output Matrix

The output y should also be organized by samples.

$$y \in \mathbb{R}^{n \times k}$$

where:

- n = number of samples
- k = output dimension

For example:

- Regression with one-dimensional output: $y \in \mathbb{R}^{n \times 1}$
- Classification with class labels: $y \in \mathbb{R}^n$
- One-hot classification: $y \in \mathbb{R}^{n \times k}$

Example: Extracting X and y from a DataFrame

Suppose the last column is the target variable.

Using pandas

```
import pandas as pd

df = pd.read_csv("data.csv")

X = df.iloc[:, :-1].values    # all columns except last
y = df.iloc[:, -1].values     # last column
```

Check the shapes:

```
print(X.shape)
print(y.shape)
```

Why Feature Scaling Matters

Neural networks are sensitive to feature scale.

- Features with large magnitudes can dominate gradient updates.
- Poor scaling can slow down optimization.
- Well-scaled inputs usually improve numerical stability.
- Scaling helps gradient-based methods converge more reliably.

Note that it is not always necessary to scale your data. It depends on the scale of your data features. If they have huge difference, e.g. $x_1 \in [0, 1]$ and $x_2 \in [1, 1000]$, then it might be necessary to scale your data first.

Standardization

Standardization transforms each feature to have approximately zero mean and unit variance.

$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

where μ and σ are computed from the training set.

- Common default choice for neural networks.
- Useful when features have different units or scales.
- Does not force values into a fixed range.
- Less sensitive to the exact minimum and maximum values.

Normalization

Normalization rescales feature values to a fixed range, usually from 0 to 1.

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

where $x_{\min}$ and $x_{\max}$ are computed from the training set.

- Useful when features have known lower and upper bounds.
- Common for image pixel values.
- Keeps all features within a comparable numerical range.
- Sensitive to outliers.

Standardization vs. Normalization

Method	Output scale	Typical use
Standardization	Mean 0, variance 1	General tabular data
Normalization	Fixed range, often 0 to 1	Bounded data, images

- Use standardization when feature distributions are unbounded or roughly Gaussian.
- Use normalization when the feature range is meaningful and bounded.

Standardization in Python

Using scikit-learn

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_val_scaled = scaler.transform(X_val)
```

```
X_test_scaled = scaler.transform(X_test)
```

- `fit_transform` learns parameters from the training set.
- `transform` applies the learned transformation.

Normalization in Python

Using scikit-learn

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))

X_train_scaled = scaler.fit_transform(X_train)
X_val_scaled = scaler.transform(X_val)
X_test_scaled = scaler.transform(X_test)
```

Use the same scaler object for training, validation, and test sets.

Note that scaling is generally applied **only to input data**. In most cases, output data is not necessary to be scaled, except in some specific regression problems where target values have unstable numerical ranges.

Training, Validation, and Testing Sets

A dataset is commonly divided into three parts:

$$\text{Dataset} = \text{Training set} + \text{Validation set} + \text{Testing set}$$

- **Training set:** used to train the model parameters
- **Validation set:** used to tune model choices and hyperparameters
- **Testing set:** used only at the end to evaluate final performance

Note that sometime two sets (training+testing) are also enough. Typically if you are confident enough that your model will not be overfitted, then validation dataset is not necessary.

Why Do We Split the Dataset?

When training a neural network, we should not evaluate it only on the data used for training.

- The model may simply memorize the training data
- We want to know whether the model can generalize to unseen data
- Data splitting helps us estimate real-world performance

Training Set

The training set is used to update the neural network weights.

- The model sees these samples during training
- Loss is computed on the training data
- Backpropagation uses the training loss to update weights

$$\theta \leftarrow \theta - \alpha \frac{\partial L_{\text{train}}}{\partial \theta}$$

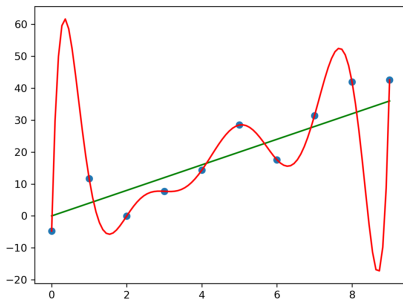
where θ represents the model parameters and α is the learning rate.

Validation Set

The validation set is used to check model performance during development. It helps detect overfitting.

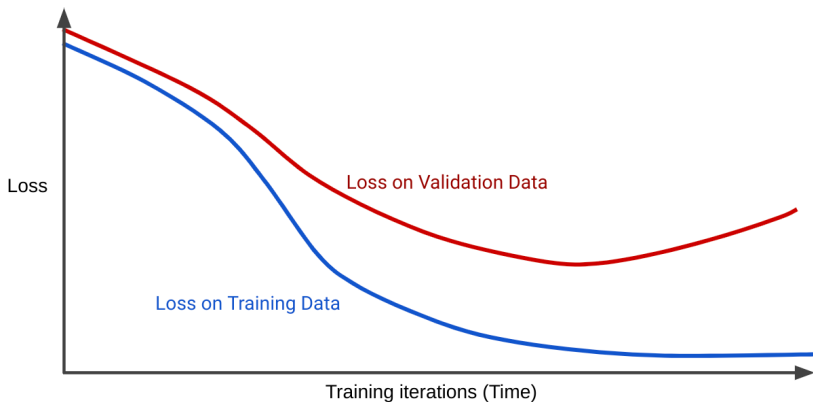
Overfitting

Overfitting happens when a model fits the training data too closely, including noise or irrelevant patterns, instead of learning the general relationship.



Validation Set

Good training performance (low loss function value) $\neq$ a good model.



A good model should perform well not only on the training data but also on unseen data, which reflects its ability to generalize.

Testing Set

The testing set is used only after the model has been trained.

- It estimates performance on unseen data
- It provides the final evaluation result

Testing set $\approx$ future unseen real-world data

Typical Split Ratios

Common dataset split ratios include:

70% training, 15% validation, 15% testing

or

80% training, 10% validation, 10% testing

The exact ratio depends on:

- dataset size
- task difficulty
- need for reliable evaluation

Training and Validation Loss

During training, we often monitor both training loss and validation loss.

L_{train} = loss on training set

L_{val} = loss on validation set

- If training loss is decreasing but validation loss is increasing: possible overfitting
- If both losses are decreasing: better generalization

Data Splitting with scikit-learn

We can use `train_test_split` to split data (works for both numpy data and pytorch data).

First split: training and temporary data

```
from sklearn.model_selection import train_test_split

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

Here:

70% → training

30% → temporary set

Creating Validation and Testing Sets

Then split the temporary set into validation and testing sets.

Second split: validation and testing

```
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=42
)
```

Final split:

70% → training

15% → validation

15% → testing

Checking the Split

After splitting, always check the shapes.

Shape checking

```
print("X_train:", X_train.shape)
print("y_train:", y_train.shape)
```

```
print("X_val:", X_val.shape)
print("y_val:", y_val.shape)
```

```
print("X_test:", X_test.shape)
print("y_test:", y_test.shape)
```

The number of rows in each X set must match the number of rows in the corresponding y set.

Two Common Ways to Build a Neural Network

In PyTorch, neural network models are usually built using `torch.nn`.

- **Method 1:** Use `nn.Sequential`

- **Method 2:** Define layers and write a `forward` function in a class

Both methods can define the same neural network structure.

Required PyTorch Modules

Import PyTorch modules

```
import torch
import torch.nn as nn
```

- `torch` provides tensor operations.
- `torch.nn` provides neural network layers, activation functions, and loss functions.

Method 1: Using nn.Sequential

`nn.Sequential` builds a model by stacking layers in order.

Example: Fully connected neural network

```
model = nn.Sequential(  
    nn.Linear(4, 16),  
    nn.ReLU(),  
    nn.Linear(16, 8),  
    nn.ReLU(),  
    nn.Linear(8, 3)  
)
```

This model maps a 4-dimensional input to a 3-dimensional output.

Understanding the Sequential Model

The model structure is:

Input dimension: 4

$4 \rightarrow 16 \rightarrow 8 \rightarrow 3$

- `nn.Linear(4, 16)` maps input features to 16 hidden units.
- `nn.ReLU()` adds nonlinearity.
- `nn.Linear(16, 8)` maps 16 hidden units to 8 hidden units.
- `nn.ReLU()` adds nonlinearity.
- `nn.Linear(8, 3)` produces the final output.

When to Use nn.Sequential

`nn.Sequential` is suitable when the model is a simple chain of layers.

- Clean and concise.
- Good for standard feedforward networks.
- Easy to read when layers are applied one after another.

It is less flexible when the model needs branching, skip connections, multiple inputs, or custom operations.

Method 2: Define a Model Class

Model class

```
class NeuralNetwork(nn.Module):  
    def __init__(self):  
        super().__init__()  
  
        self.fc1 = nn.Linear(4, 16)  
        self.relu1 = nn.ReLU()  
        self.fc2 = nn.Linear(16, 8)  
        self.relu2 = nn.ReLU()  
        self.fc3 = nn.Linear(8, 3)  
  
    def forward(self, x):  
        x = self.fc1(x)  
        x = self.relu1(x)  
        x = self.fc2(x)  
        x = self.relu2(x)  
        x = self.fc3(x)  
        return x
```


Creating and Using the Model Class

After defining the class, create a model object.

Create model and run forward pass

```
model = NeuralNetwork()
```

```
x = torch.randn(5, 4)
```

```
output = model(x)
```

```
print(output.shape)
```

Calling `model(x)` automatically executes the `forward` function.

Role of `__init__` and `forward`

A PyTorch model class usually has two important parts.

- `__init__`: defines the layers and model components.
- `forward`: defines how data flows through the model.

The layers are created once in `__init__`.

The computation order is specified in `forward`.

Sequential vs. Custom Class

Method	Advantage	Best for
<code>nn.Sequential</code>	Simple and compact	Straight layer stacks
Custom class	Flexible and explicit	Complex architectures

- Use `nn.Sequential` for simple feedforward models.
- Use a custom class when the forward computation is not purely sequential.

Input and Output Dimensions

The first linear layer must match the input feature dimension.

```
nn.Linear(input_dim, hidden_dim)
```

The next linear layer must match the output dimension of the previous layer.

```
nn.Linear(hidden_dim, output_dim)
```

Examples:

- Binary classification: output dimension can be 1.
- Multi-class classification with 3 classes: output dimension is 3.
- Regression with 2 target values: output dimension is 2.

Neural Network Training: Define Loss Function and Optimizer

The loss function measures prediction error.

The optimizer updates model parameters using gradients.

Example

```
import torch.nn as nn
import torch.optim as optim

criterion = nn.MSELoss()

optimizer = optim.Adam(
    model.parameters(),
    lr=0.001
)
```

`model.parameters()` tells the optimizer which trainable parameters to update.

Common Choices

The loss function depends on the task.

Task	Common loss function
Regression	<code>nn.MSELoss()</code>
Binary classification	<code>nn.BCEWithLogitsLoss()</code>
Multi-class classification	<code>nn.CrossEntropyLoss()</code>

Common optimizers:

- `optim.SGD`
- `optim.Adam`
- `optim.AdamW`

Minimum Training Loop

Training loop

```
num_epochs = 100

for epoch in range(num_epochs):
    model.train()

    optimizer.zero_grad()

    y_pred = model(X_train)

    loss = criterion(y_pred, y_train)

    loss.backward()

    optimizer.step()

    print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

Meaning of Each Step

- `model.train()` enables training behavior.
- `optimizer.zero_grad()` clears old gradients.
- `model(X_train)` computes predictions.
- `criterion(y_pred, y_train)` computes the loss.
- `loss.backward()` computes gradients by backpropagation.
- `optimizer.step()` updates model parameters.
- `loss.item()` converts the loss tensor to a Python number.

Why Zero the Gradients?

PyTorch accumulates gradients by default. If gradients are not cleared before each update, new gradients are added to old gradients.

wrong gradient update = old gradients + new gradients

Therefore, call:

```
optimizer.zero_grad()
```

before computing the next backward pass.

Each training iteration has three core operations.

- **Forward pass:** compute model predictions.
- **Backward pass:** compute gradients of the loss.
- **Parameter update:** use the optimizer to update weights.

Basic Prediction Workflow

After training, the model can be used to make predictions on validation, test, or new unseen data.

Prediction

```
model.eval()
```

```
with torch.no_grad():  
    outputs = model(X_test)
```

- `model.eval()` sets the model to evaluation mode.
- `torch.no_grad()` reduces memory usage and computation.

Neural Network Implementation Summary

1 Data preparation

- Load data from files, pandas, NumPy, or existing datasets.
- Understand the dataset description, feature and target meanings
- Extract input features as x and output targets as y .
- Split data into training, validation, and testing sets.
- Scale input features using standardization or normalization when needed.

2 Model preparation

- Build the PyTorch model using `nn.Sequential` or a custom `nn.Module`.

3 Training

- Define the loss function and optimizer.
- For each epoch: zero gradients, predict, compute loss, call `loss.backward()`, then `optimizer.step()`.
- Monitor training loss and optionally validation loss.

4 Prediction

- Set `model.eval()` and use `torch.no_grad()`.
- Run the forward pass on validation, test, or new data.